

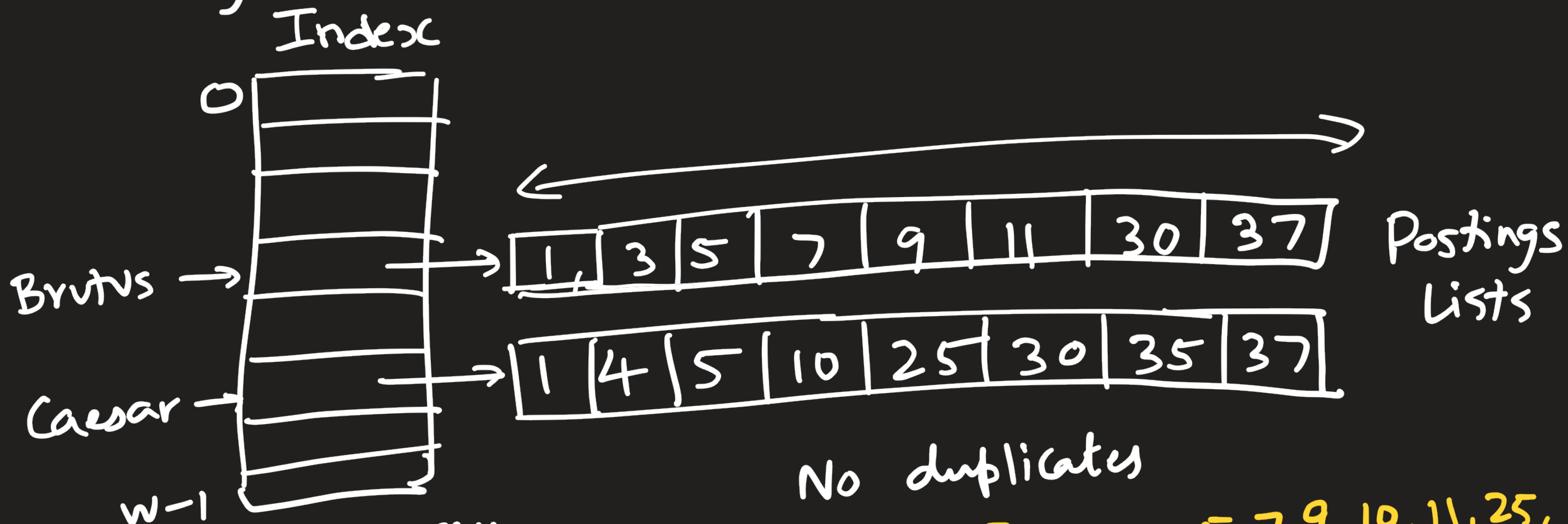
Part 2



Search index - STATIC 1-37

Query: Brutus

Result: [1, 10, 23]

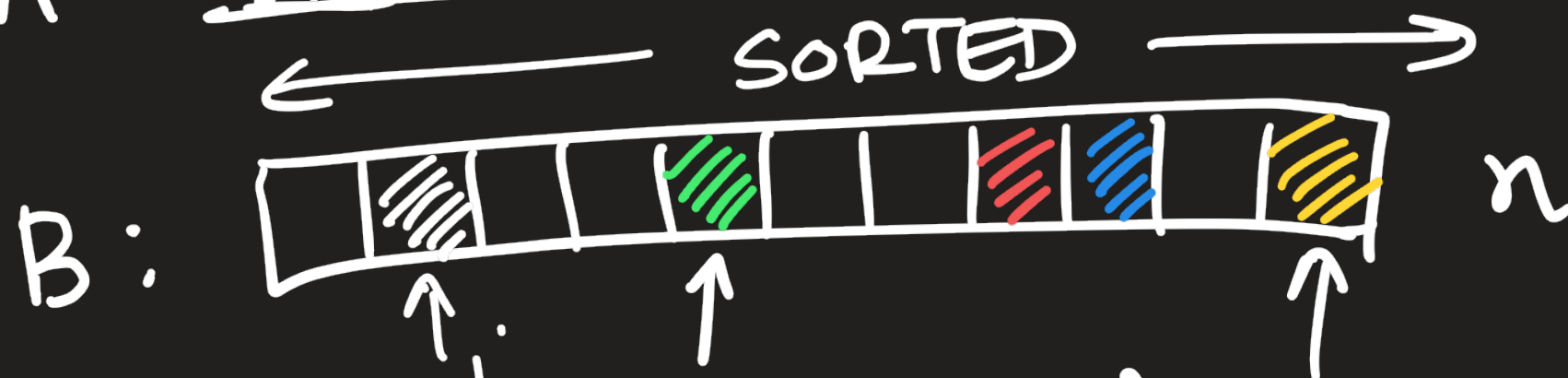


Boolean queries:

AND query → Brutus AND Caesar: [1, 5, 30, 37]

OR query → Brutus OR Caesar → Union of two sorted arrays.

Brute force: Exhaustively look at all numbers



$$T(m, n) = O(mn)$$

Decrease-and-conquer:

$$T(m, n) = O(m \log n) \\ \text{or } O(n \log m)$$

m binary searches
on larger array

$\rightarrow O(c \log n)$ if m is const.
 $= O(\log n)$

Two POINTER PASS :

$$T(m, n) = O(m+n)$$

Hash Table (Transform & Conquer) :

Insert into hash table $\rightarrow O(n)$

m search ops in the hash table $\rightarrow O(m)$

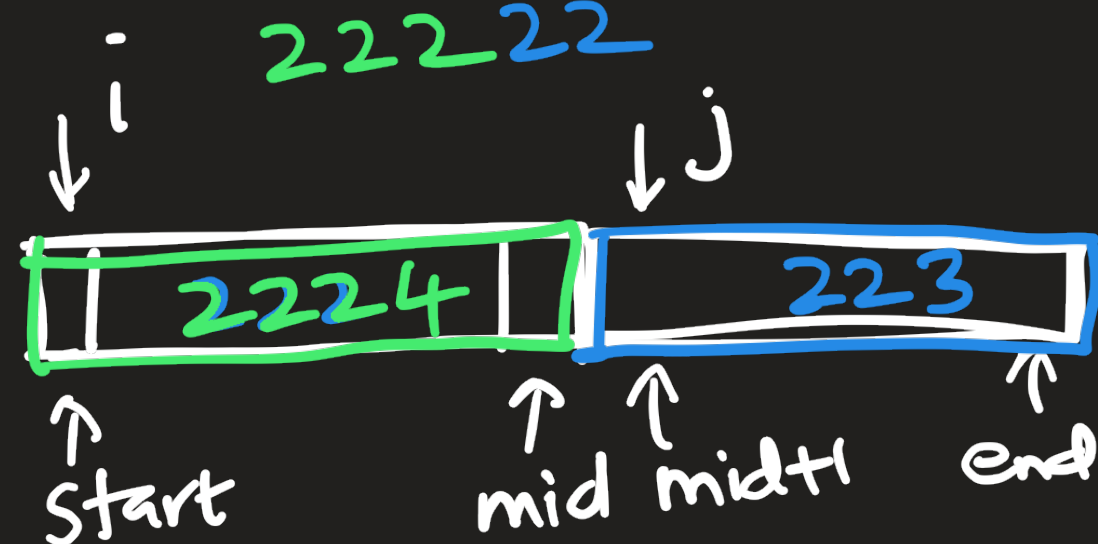
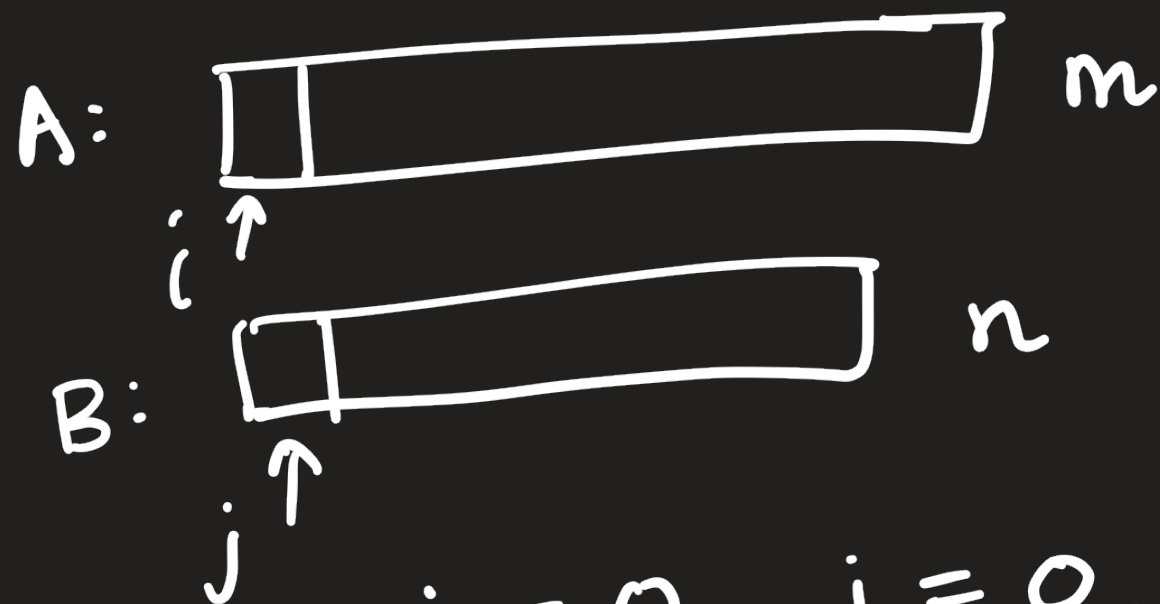
$O(m+n)$ total, with $O(n)$ aux space

Smaller array

What if one of the arrays was v. Small?

$m = C$ Switch to the

$\rightarrow$ Binary search approach



$i = 0, j = 0, \text{result} = []$

while $i \leq m-1$ and $j \leq n-1$:

if $A[i] == B[j]$:
 $\text{result.append}(A[i])$
 $i++, j++$

else if $A[i] < B[j]$:
 $i++ \leftarrow \text{result.append}(A[i])$

else: // $A[i] > B[j]$
 $j++ \leftarrow \text{result.append}(B[j])$

// GATHER PHASE

return result

$i = \text{start}, j = \text{mid} + 1$
 $\text{aux} = []$

// Gather phase

while $i \leq m-1$:

{
 result.append($A[i]$)
 $i++$
}

while $j \leq n-1$:

{
 result.append($B[j]$)
 $j++$
}

1213. Intersection of Three Sorted Arrays

Easy

👍 37

💬 3

♡ Favorite

📄 Share

Given three integer arrays `arr1`, `arr2` and `arr3` **sorted** in **strictly increasing** order, return a sorted array of **only** the integers that appeared in **all** three arrays.

Example 1:

Input: `arr1 = [1,2,3,4,5]`, `arr2 = [1,2,5,7,9]`, `arr3 = [1,3,4,5,8]`

Output: `[1,5]`

Explanation: Only 1 and 5 appeared in the three arrays.

Constraints:

- `1 <= arr1.length, arr2.length, arr3.length <= 1000`
- `1 <= arr1[i], arr2[i], arr3[i] <= 2000`

```
10 i = 0
11 j = 0
12 k = 0
13 result = []
14 ▼ while i < len(arr1) and j < len(arr2) and k < len(arr3):
15 ▼     if arr1[i] == arr2[j] == arr3[k]:
16         result.append(arr1[i])
17         i += 1
18         j += 1
19         k += 1
20 ▼     elif arr1[i] <= arr2[j] and arr1[i] <= arr3[k]:
21         i += 1
22 ▼     elif arr2[j] <= arr1[i] and arr2[j] <= arr3[k]:
23         j += 1
24 ▼     else:
25         k += 1
26
27 return result
```

349. Intersection of Two Arrays

Easy

👍 869

💬 1251

♡ Add to List

📄 Share

Given two arrays, write a function to compute their intersection.

Example 1:

Input: nums1 = [1,2,2,1], nums2 = [2,2]
Output: [2]

Example 2:

Input: nums1 = [4,9,5], nums2 = [9,4,9,8,4]
Output: [9,4]

Note:

- Each element in the result must be unique.
- The result can be in any order.

Space-Time
tradeoff between
presorting &
using hash
tables.

88. Merge Sorted Array

Easy

👍 1275

💬 3041

♡ Favorite

🔗 Share

Given two sorted integer arrays *nums1* and *nums2*, merge *nums2* into *nums1* as one sorted array.

Note:

- The number of elements initialized in *nums1* and *nums2* are *m* and *n* respectively.
- You may assume that *nums1* has enough space (size that is greater or equal to $m + n$) to hold additional elements from *nums2*.

Example:

Input:

`nums1 = [1,2,3,0,0,0], m = 3`

`nums2 = [2,5,6], n = 3`

Output: `[1,2,2,3,5,6]`

Part 3



215. Kth Largest Element in an Array

Medium

👍 2229

💬 183

♡ Favorite

🔗 Share

Find the **k**th largest element in an unsorted array. Note that it is the kth largest element in the sorted order, not the kth distinct element.

Example 1:

Input: [3,2,1,5,6,4] and k = 2

Output: 5

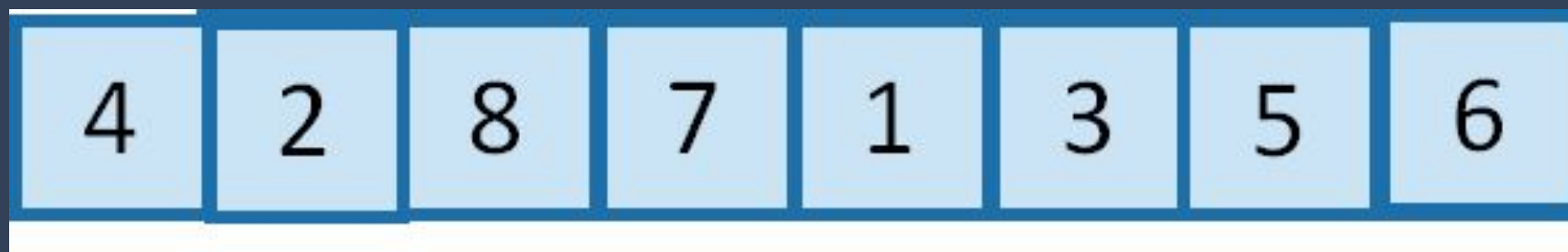
Example 2:

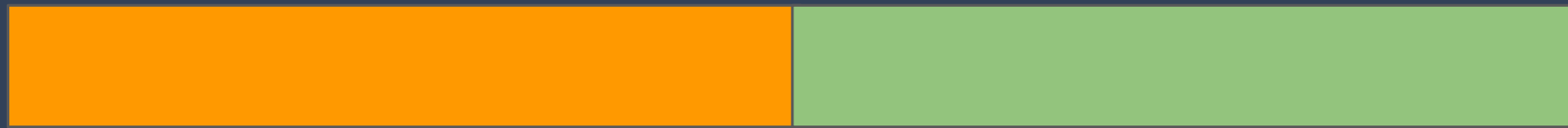
Input: [3,2,3,1,2,4,5,5,6] and k = 4

Output: 4

Note:

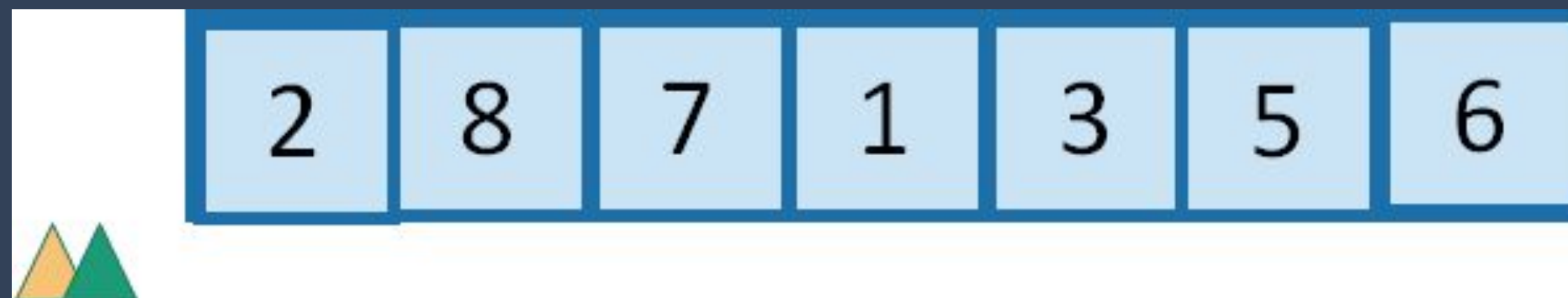
You may assume k is always valid, $1 \leq k \leq \text{array's length}$.





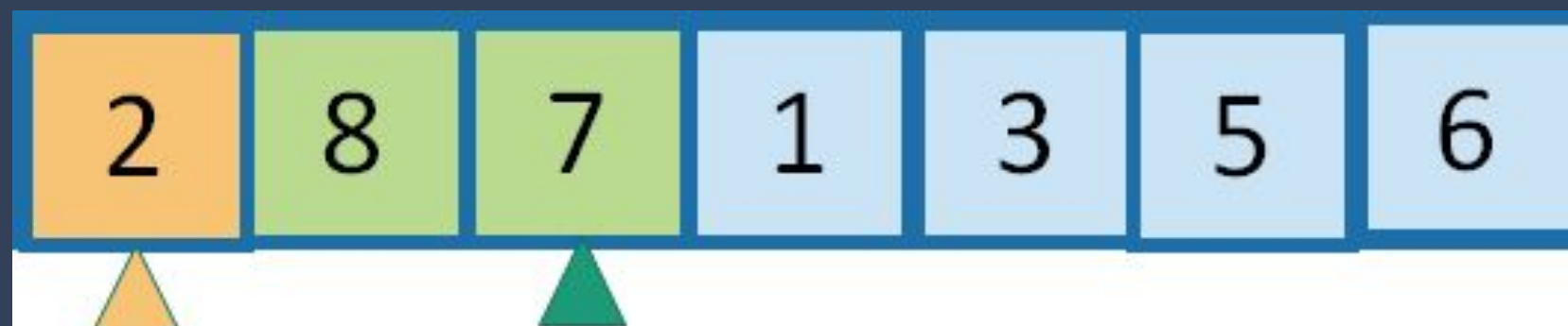
< value

> value



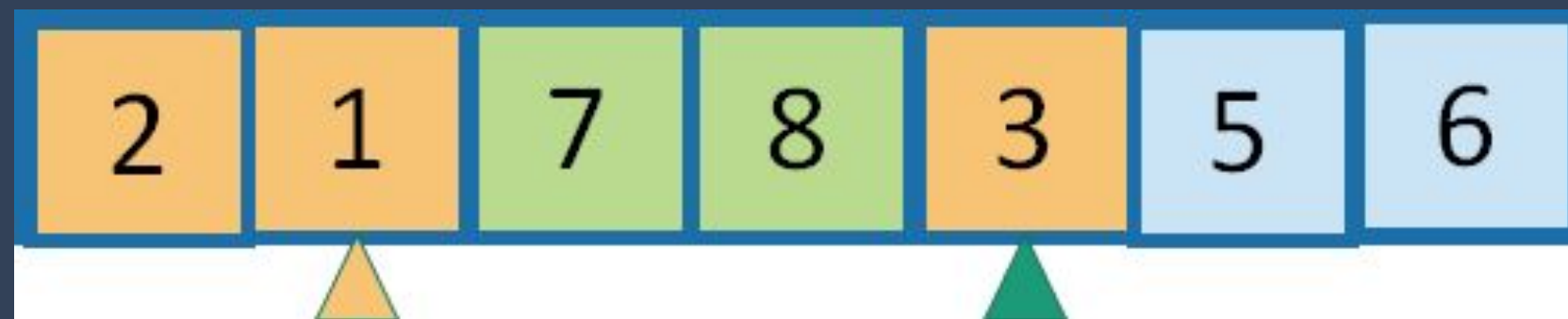


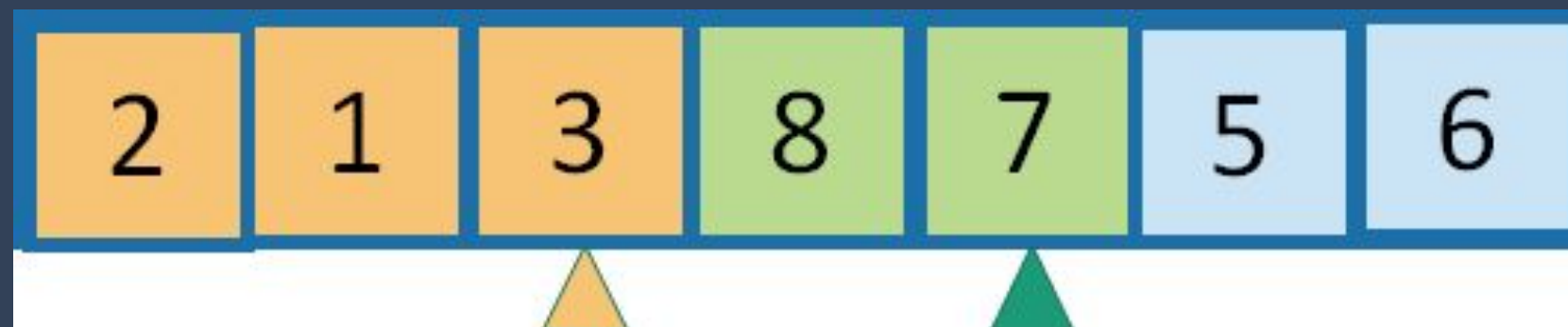


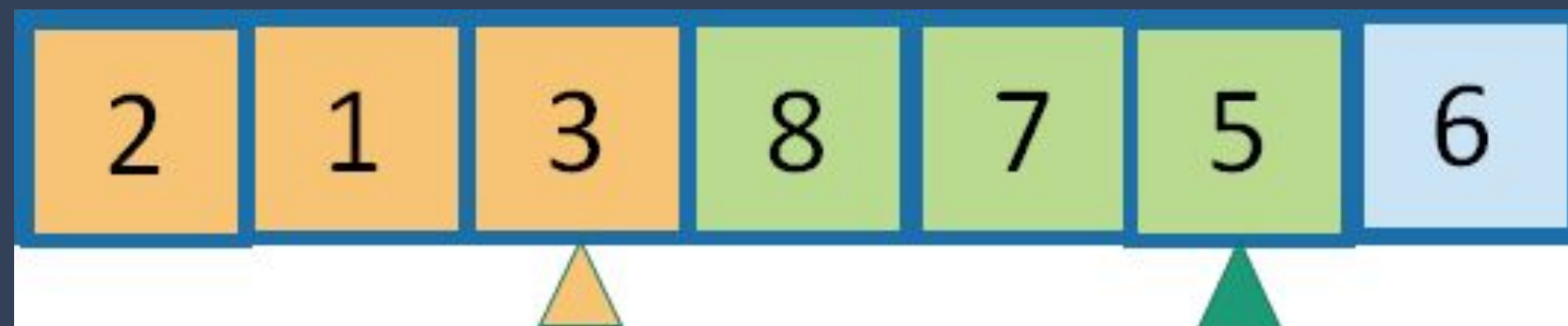


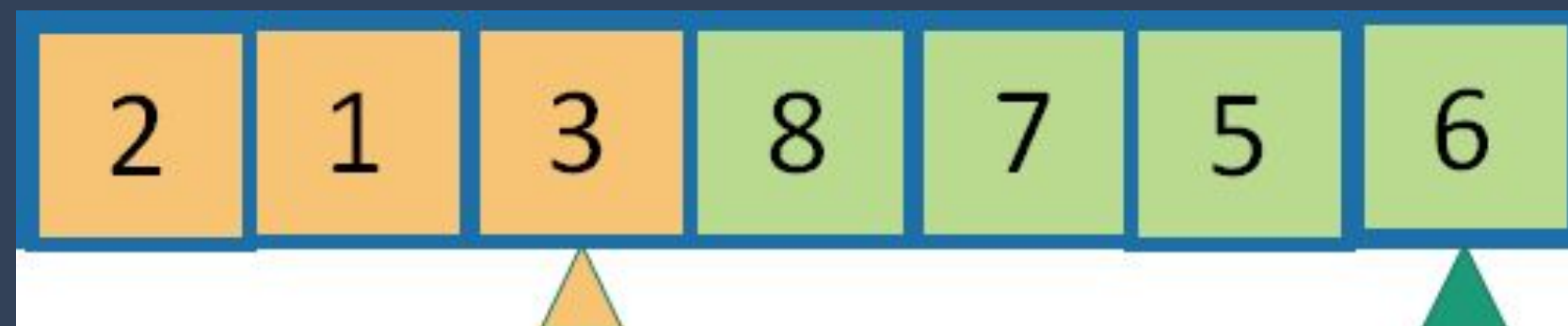


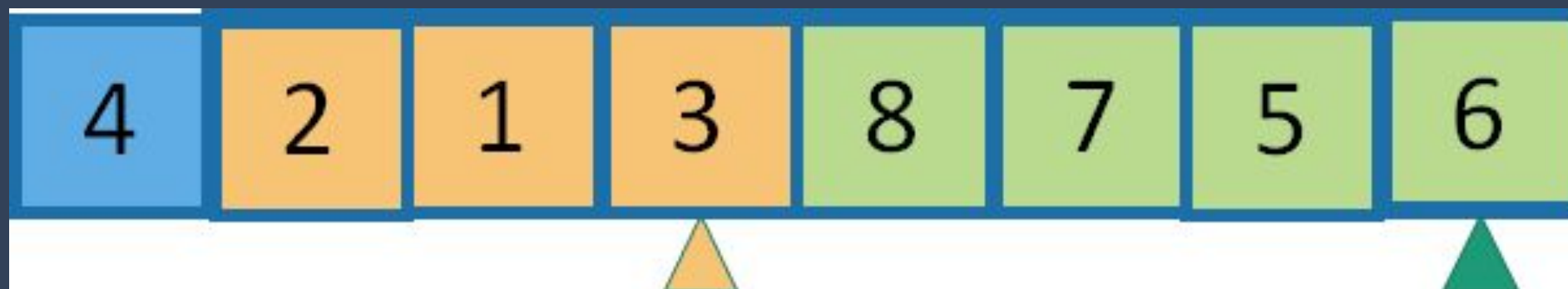


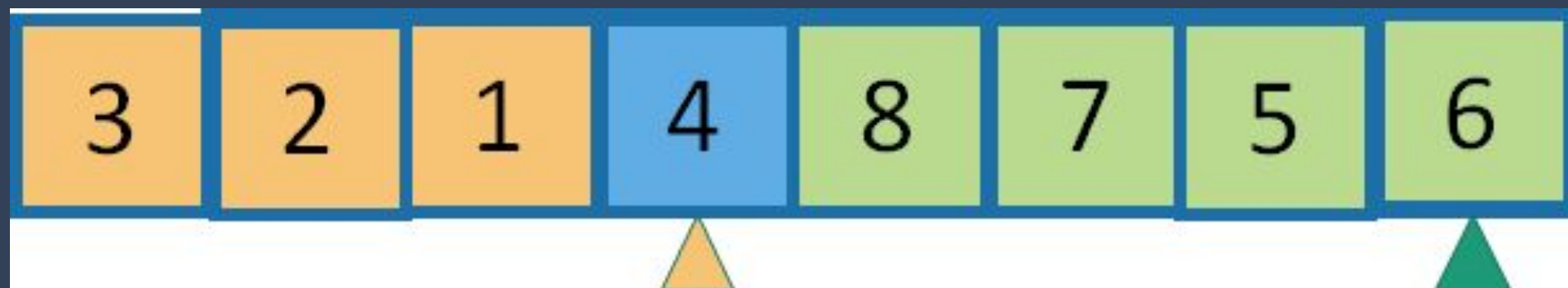












Lomuto's partitioning: $O(n)$ time, in place

Best case?

Worst case?

Average case?

973. K Closest Points to Origin

Medium

👍 547

💬 56

♡ Favorite

🔗 Share

We have a list of `points` on the plane. Find the `K` closest points to the origin `(0, 0)`.

(Here, the distance between two points on a plane is the Euclidean distance.)

You may return the answer in any order. The answer is guaranteed to be unique (except for the order that it is in.)

Example 1:

Input: `points = [[1,3],[-2,2]]`, `K = 1`

Output: `[[-2,2]]`

Explanation:

The distance between `(1, 3)` and the origin is `sqrt(10)`.

The distance between `(-2, 2)` and the origin is `sqrt(8)`.

Since `sqrt(8) < sqrt(10)`, `(-2, 2)` is closer to the origin.

We only want the closest `K = 1` points from the origin, so the answer is just `[[-2,2]]`.



347. Top K Frequent Elements

Medium

👍 2130

💬 140

♡ Favorite

🔗 Share

Given a non-empty array of integers, return the ***k*** most frequent elements.

Example 1:

Input: `nums = [1,1,1,2,2,3], k = 2`

Output: `[1,2]`

Example 2:

Input: `nums = [1], k = 1`

Output: `[1]`

Note:

- You may assume *k* is always valid, $1 \leq k \leq$ number of unique elements.
- Your algorithm's time complexity **must be** better than $O(n \log n)$, where *n* is the array's size.

Solution

Intuition

If $k = 1$ the linear-time solution is quite simple. One could keep the frequency of elements appearance in a hash map and update the maximum element at each step.

When $k > 1$ we need a data structure that has a fast access to the elements ordered by their frequencies. The idea here is to use the heap which is also known as priority queue.

Approach 1: Heap

The first step is to build a hash map `element -> its frequency`. In Java we could use data structure `HashMap` but have to fill it manually. Python provides us both a dictionary structure for the hash map and a method `Counter` in the `collections` library to build the hash map we need. This step takes $\mathcal{O}(N)$ time where N is number of elements in the list.

The second step is to build a heap. The time complexity of adding an element in a heap is $\mathcal{O}(\log(k))$ and we do it N times that means $\mathcal{O}(N \log(k))$ time complexity for this step.

The last step to build an output list has $\mathcal{O}(k \log(k))$ time complexity.

In Python there is a method `nlargest` in `heapq` library ([check here the source code](#)) which has the same $\mathcal{O}(k \log(k))$ time complexity and combines two last steps in one line.

Complexity Analysis

- Time complexity : $\mathcal{O}(N \log(k))$. The complexity of `Counter` method is $\mathcal{O}(N)$. To build a heap and output list takes $\mathcal{O}(N \log(k))$. Hence the overall complexity of the algorithm is $\mathcal{O}(N + N \log(k)) = \mathcal{O}(N \log(k))$.
- Space complexity : $\mathcal{O}(N)$ to store the hash map.

Side Notes

Following the complexity analysis, the approach is optimal for small `k`. In the case of large `k`, one could revert the procedure by excluding the less frequent elements from the output.

Analysis written by @liaison and @andvary



JPV



515

December 28, 2018 12:41 PM

The problem specifies that:

"Your algorithm's time complexity must be better than $O(n \log n)$, where n is the array's size."

LeetCode, why do you have an official solution that fails to meet your own requirement? Please either post a solution that meets the requirement, or change the problem description.

(And, no, $O(n \log k)$ is not better than $O(n \log n)$, since we have no reason to think k isn't, say, $n/2$.)



67



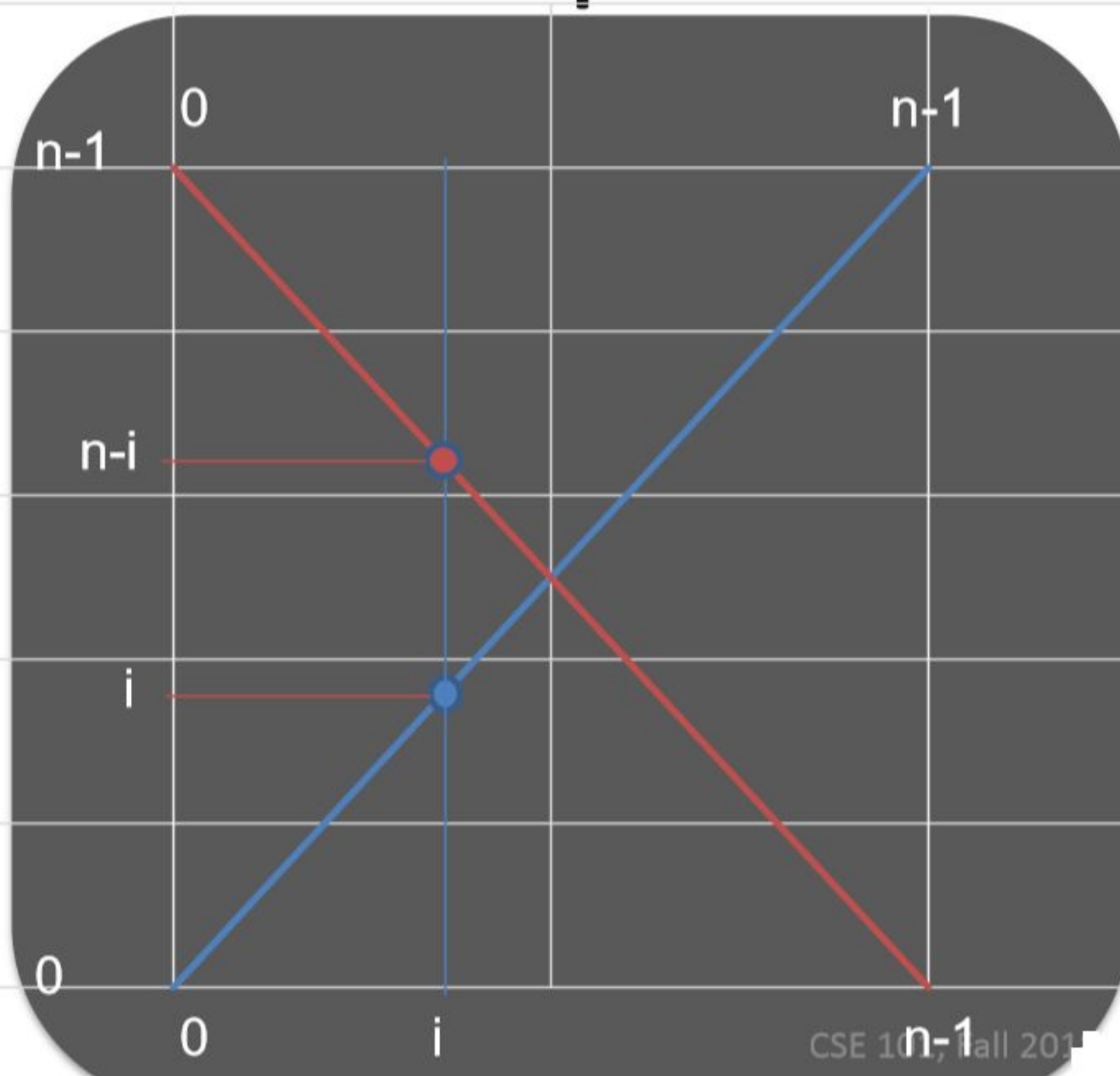
Show 12 replies



Reply

Update: Leetcode recently fixed the issue

Expected runtime

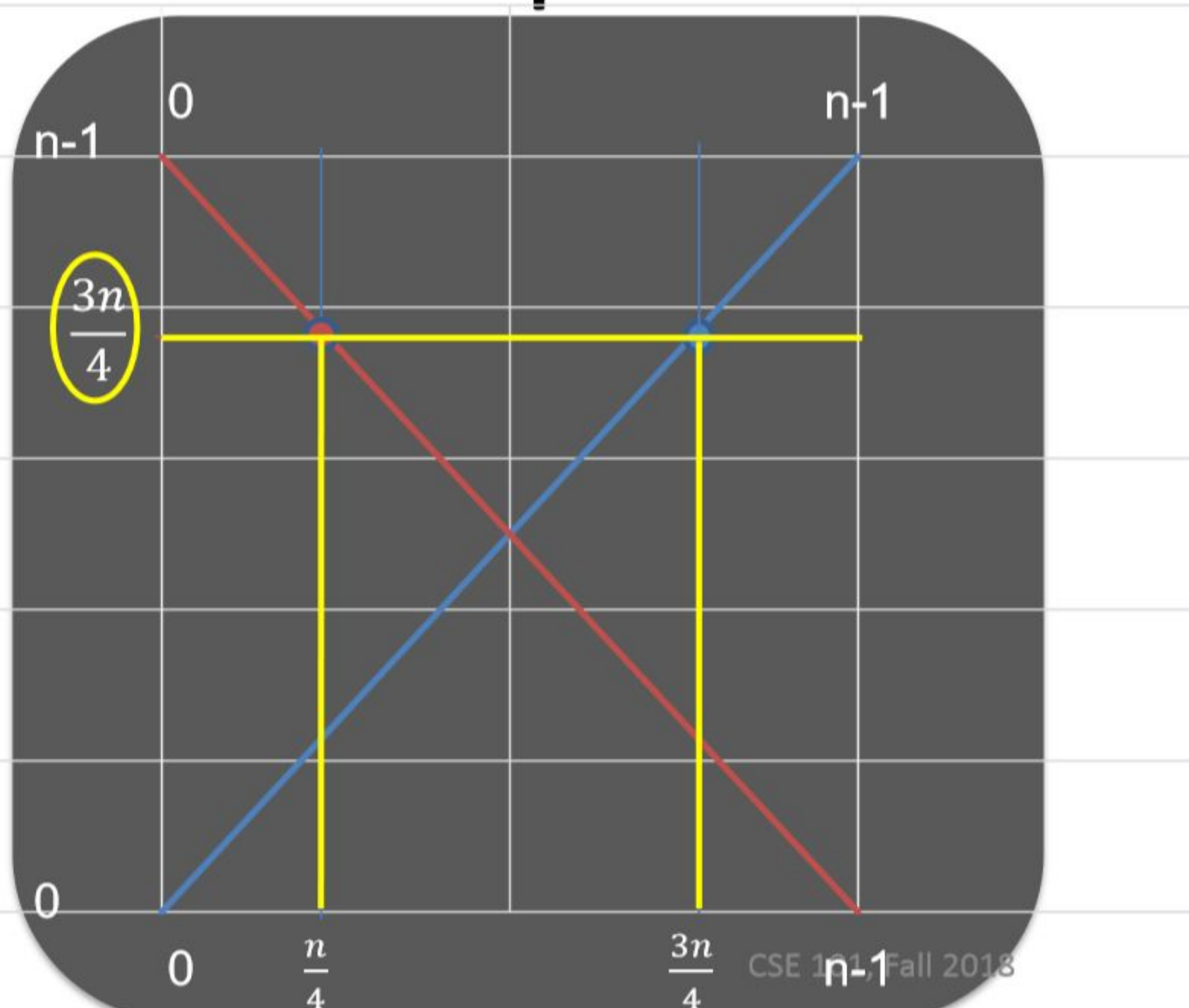


If you randomly select the i th element, then your list will be split into a list of length i and a list of length $n-i$

So when we recurse on the smaller list, it will take time proportional to

$$\max(i, n - i)$$

Expected runtime



What is the probability of choosing a value from 1 to n in the interval $\left[\frac{n}{4}, \frac{3n}{4}\right]$ if all values are equally likely?

Median

- The purpose of the median is to summarize a set of numbers. The average is also a commonly used value. The median is more typical of the data.
- For example, suppose in a company with 20 employees, the CEO makes \$1 million and all the other workers each make \$50,000
- Then the average is \$97,500 and the median is \$50,000, which is much closer to the typical worker's salary

Median, algorithm

- Can you think of an efficient way to find the median?
 - How long would it take?
 - Is there a lower bound on the runtime of all median selection algorithms?

Median, algorithm

- Can you think of an efficient way to find the median?
 - How long would it take?
 - Is there a lower bound on the runtime of all median selection algorithms?
- Sort the list then find the $[n/2]$ th element $O(n \log n)$
- You can never have a faster runtime than $O(n)$ because you at least have to look at every element

75. Sort Colors

Medium

👍 3594

💬 229

♡ Add to List

🔗 Share

Given an array with n objects colored red, white or blue, sort them **in-place** so that objects of the same color are adjacent, with the colors in the order red, white and blue.

Here, we will use the integers 0, 1, and 2 to represent the color red, white, and blue respectively.

Note: You are not suppose to use the library's sort function for this problem.

Example:

Input: [2,0,2,1,1,0]

Output: [0,0,1,1,2,2]

Follow up:

- A rather straight forward solution is a two-pass algorithm using counting sort.
First, iterate the array counting number of 0's, 1's, and 2's, then overwrite array with total number of 0's, then 1's and followed by 2's.
- Could you come up with a one-pass algorithm using only constant space?



Part 4

703. Kth Largest Element in a Stream

Easy  351  164  Favorite  Share

Design a class to find the **k**th largest element in a stream. Note that it is the kth largest element in the sorted order, not the kth distinct element.

Your `KthLargest` class will have a constructor which accepts an integer `k` and an integer array `nums`, which contains initial elements from the stream. For each call to the method `KthLargest.add`, return the element representing the kth largest element in the stream.

Example:

```
int k = 3;
int[] arr = [4,5,8,2];
KthLargest kthLargest = new KthLargest(3, arr);
kthLargest.add(3);    // returns 4
kthLargest.add(5);    // returns 5
kthLargest.add(10);   // returns 5
kthLargest.add(9);    // returns 8
kthLargest.add(4);    // returns 8
```

Note:

You may assume that `nums`' length $\geq k-1$ and $k \geq 1$.

Write a program that takes a sequence of strings in “streaming” fashion, and computes the k longest strings in the sequence.

Time complexity after seeing n strings = ?

Write a program that takes a sequence of strings in “streaming” fashion, and computes the k longest strings in the sequence.

```
minheap = []
```

```
for s in sequence:
```

```
    minheap.insert(s,priority=len(s))
```

```
    if len(minheap) > k:
```

```
        minheap.extractmin()
```

```
//minheap always maintains the k longest strings at any time
```

```
1 ▼ class KthLargest(object):
2
3 ▼     def __init__(self, k, nums):
4         """
5         :type k: int
6         :type nums: List[int]
7         """
8         self.minheap = nums
9         self.k = k
10        heapq.heapify(self.minheap)
11 ▼        while len(self.minheap) > k:
12            heapq.heappop(self.minheap)
13
14
15 ▼     def add(self, val):
16         """
17         :type val: int
18         :rtype: int
19         """
20        heapq.heappush(self.minheap, val)
21 ▼        if len(self.minheap) > self.k:
22            heapq.heappop(self.minheap)
23        return self.minheap[0]
24
```

A heap is a good choice when you are maintaining the k largest or k smallest elements in any collection.

But what about the median finding problem?

Compute the Median of Online data

Compute the running median of a sequence of numbers. The sequence is presented to you in a streaming fashion - you cannot back up to read an earlier value, and you need to output the median after reading in each new element.

Example input: 1,0,3,5,2,0,1

Output: 1, 0.5, 1, 2, 2, 1.5, 1

295. Find Median from Data Stream

Hard  1306  25  Favorite  Share

Median is the middle value in an ordered integer list. If the size of the list is even, there is no middle value. So the median is the mean of the two middle value.

For example,

[2,3,4] , the median is 3

[2,3] , the median is $(2 + 3) / 2 = 2.5$

Design a data structure that supports the following two operations:

- void addNum(int num) - Add a integer number from the data stream to the data structure.
- double findMedian() - Return the median of all elements so far.

Example:

```
addNum(1)
addNum(2)
findMedian() -> 1.5
addNum(3)
findMedian() -> 2
```

```

1 class MedianFinder(object):
2
3     def __init__(self):
4         """
5         initialize your data structure here.
6         """
7         self.minheap = []
8         self.maxheap = []
9         self.median = 0.0
10
11
12     def addNum(self, num):
13         """
14         :type num: int
15         :rtype: None
16         """
17         if num > self.median:
18             heapq.heappush(self.minheap, num)
19             if len(self.minheap) - len(self.maxheap) == 2:
20                 heapq.heappush(self.maxheap, -heapq.heappop(self.minheap))
21         else:
22             heapq.heappush(self.maxheap, -num)
23             if len(self.maxheap) - len(self.minheap) == 2:
24                 heapq.heappush(self.minheap, -heapq.heappop(self.maxheap))
25
26         if len(self.minheap) == len(self.maxheap):
27             self.median = (self.minheap[0] - self.maxheap[0])/2.0
28         elif len(self.minheap) > len(self.maxheap):
29             self.median = self.minheap[0]
30         else:
31             self.median = -self.maxheap[0]
32
33
34     def findMedian(self):
35         """
36         :rtype: float
37         """
38         return self.median

```